SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

PHASES OF COMPILER AND THEIR ROLE

By:

Varunkumar B

192525029

Supervisor:

Dr G Michael

Department of CSE

Introduction

- **What is a Compiler?**
- Translates a high-level programming language into machine code.
- Checks the source program for syntax and semantic errors.
- Optimizes the code to improve efficiency.
- Generates executable machine code for the target system.

Phases of Compiler

- • Lexical Analysis – Converts characters into tokens.
- • Syntax Analysis – Checks grammar.
- • Semantic Analysis – Checks meaning.
- • Intermediate Code Generation.
- • Code Optimization.
- • Code Generation.

Lexical Analysis (Scanner)

- Reads source code character by character.
- Groups characters into meaningful tokens.
- Removes comments and unnecessary spaces.
- Passes the token stream on to the parser.

Syntax Analysis (Parser)

- Checks whether the token stream follows grammar rules.
- Builds a Parse Tree or Abstract Syntax Tree (AST).
- Reports syntax errors to the programmer

Semantic Analysis

Semantic Analysis

Confirms every identifier used has been declared.

Type Compatibility

Verifies operations use compatible data types.

Function Arguments

Matches call arguments against parameter lists.

Scope Rules

Validates visibility and lifetime of identifiers.

Intermediate Code Generation

- Converts the source program into an Intermediate Representation (IR).
- Makes the compiler machine-independent.
- Sits between the front end and back end of the compiler.
- rce Program

Code Optimization

- Reduce execution time.
- Reduce memory usage.
- Remove unnecessary instructions.
- Constant folding
- Dead code elimination
- Common subexpression elimination
- Loop optimization

Code Generation

- Generates target machine code from the intermediate representation.
- The final phase — its output is directly executable.
- Register allocation
- Instruction selection
- Memory management
- Machine code generation

Advantages & Applications

- **Advantages:**
- • Fast token generation
- • Easy maintenance
- • Reduces manual coding

- **Applications:**
- • Compilers
- • Interpreters
- • IDEs
- • Syntax highlighting

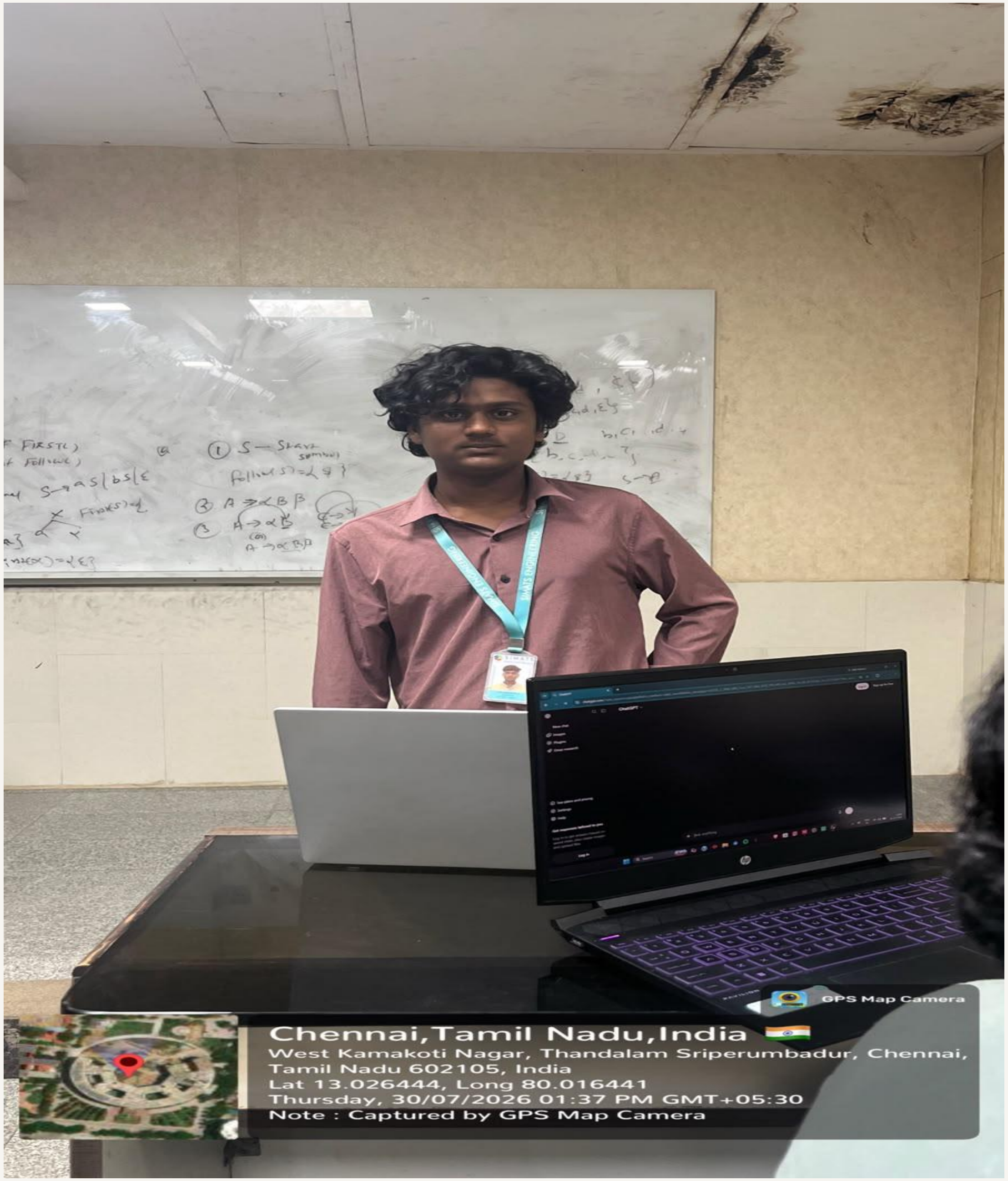
Limitation

- Increased overall compilation time compared to a single-pass design.
- Higher memory overhead from maintaining IR and symbol table data across phases.
- Design and implementation complexity grows with each additional phase.
- An error in one phase can propagate and surface as confusing errors later.
- Some optimizations require information that is easier to lose across phase boundaries.

Conclusion

- A compiler translates high-level code into machine code through multiple well-defined phases.
- Each phase performs a specific task, contributing to the overall correctness and efficiency of the compiled program.
- Supporting components — the Symbol Table and Error Handler — are used throughout the compilation process.
- Understanding compiler phases is essential for designing efficient programming language translators.

THANK
YOU


GPS Map Camera
Chennai,Tamil Nadu,India
West Kamakoti Nagar, Thandalam Sriperumbadur, Chennai, Tamil Nadu 602105, India
Lat 13.026444, Long 80.016441
Thursday, 30/07/2026 01:37 PM GMT+05:30
Note : Captured by GPS Map Camera